

PET TERRITORY WARS

Backend Architecture

Version 1.1

Go tabanlı API, worker süreçleri, PostgreSQL ve outbox altyapısı için bağlayıcı mimari kurallar

Status: Approved for Milestone 0 implementation

Document owner: Pet Territory Wars Engineering

Language: Turkish • Technical identifiers: English

Belge Kontrolü

Belge	Pet Territory Wars — Backend Architecture v1.1
Sürüm	1.1
Durum	Approved
Kapsam	Backend runtime, API, workers, database, outbox ve operasyonel sınırlar
Ana teknoloji	Go, PostgreSQL/PostGIS, pgx/pgxpool
Öncelik sırası	Bu belge > görev dosyası olmayan varsayımlar; görev dosyası açıkça daha dar kapsam belirleyebilir

İçindekiler

1. Amaç ve kapsam
2. Mimari ilkeler
3. Runtime topolojisi
4. Kaynak kod organizasyonu
5. Configuration sözleşmesi
6. PostgreSQL bağlantı mimarisi
7. HTTP API sınırları
8. Worker mimarisi
9. Outbox modeli
10. Logging ve observability
11. Hata modeli
12. Yaşam döngüsü ve graceful shutdown
13. Güvenlik
14. Test stratejisi
15. Migration ve veri erişimi
16. Bağımlılık kuralları
17. Milestone 0 bağlayıcı kararları
18. Kabul kontrol listesi

1. Amaç ve Kapsam

Bu belge Pet Territory Wars backend sisteminin uygulama sınırlarını, süreç topolojisini, bağımlılık yönünü ve operasyonel davranışını tanımlar. Amaç, görev bazlı geliştirme sırasında mimari kararların tahmin edilmesini engellemek ve tüm backend bileşenlerinin aynı sözleşmeye göre ilerlemesini sağlamaktır.

- API, walk-worker, outbox-worker ve simulator ayrı çalıştırılabilir Go binary'leridir.
- PostgreSQL/PostGIS sistemin kalıcı veri kaynağıdır.
- pgx/pgxpool PostgreSQL erişiminin standart sürücüsüdür.
- Configuration, logging, database ve runtime yaşam döngüsü infrastructure katmanında bulunur.
- Domain ve application katmanları infrastructure ayrıntılarına bağımlı olamaz.

Kapsam dışı: Bu sürüm frontend mimarisini, oyun tasarımını, production Kubernetes topolojisini ve ayrıntılı domain kurallarını tanımlamaz.

2. Mimari İlkeler

Tek sorumluluk: Her paket ve runtime yalnızca kendi rolünü üstlenir.

Açık bağımlılıklar: Global singleton, package init ile bağlantı açma ve gizli service locator kullanılmaz.

Fail fast: Zorunlu configuration ve kritik başlangıç bağımlılıkları startup sırasında doğrulanır.

Graceful shutdown: Süreçler sinyali alır, yeni işi durdurur, mevcut işi sınırlandırılmış sürede tamamlar ve kaynakları kapatır.

At-least-once worker semantiği: Worker işlemleri tekrar çalıştırılabilir veya idempotent tasarlanır.

Secret güvenliği: Secret değerleri loglanmaz, kaynak koda veya Compose dosyasına sabit yazılmaz.

Dar görev kapsamı: Bir milestone görevi, sonraki görevlerin uygulamasını önceden gerçekleştirmez.

3. Runtime Topolojisi

Konu	Karar	Gerekçe
api	HTTP isteklerini kabul eden uzun ömürlü süreç	Transport ve uygulama use-case'lerini barındırır; background job çalıştırmaz.
walk-worker	Yürüyüş/konum tabanlı async işleri tüketen worker	API'den bağımsız ölçeklenir ve ayrı concurrency sınırı kullanır.
outbox-worker	Transactional outbox kayıtlarını yayımlayan worker	Veritabanı transaction'ı ile dış olay yayımını ayırıştırır.

simulator	Yerel geliştirme ve kontrollü simülasyon binary'si	Production servis topolojisinin zorunlu parçası değildir.
postgres	PostgreSQL 16 + PostGIS 3.4	Kalıcı veri ve coğrafi sorgular için tek source of truth.

```

backend/
  cmd/
    api/
      walk-worker/
      outbox-worker/
      simulator/
  internal/
    domain/
    application/
    infrastructure/
    transport/
  migrations/
  testdata/

```

4. Kaynak Kod Organizasyonu

Konu	Karar	Gerekçe
internal/domain	Entity, value object, domain error ve saf domain davranışı	Go stdlib dışında bağımlılık minimum olmalı; infrastructure import edemez.
internal/application	Use-case orchestration ve port/interface tanımları	Domain'e bağımlıdır; concrete DB/HTTP ayrıntısı içermez.
internal/infrastructure	Config, database, logging, repository adapter'ları ve dış sistem adapter'ları	Application portlarını uygular.
internal/transport	HTTP handler, request/response DTO ve middleware	Application use-case'lerini çağırır; doğrudan SQL çalıştırmaz.
cmd/*	Composition root	Config yükler, bağımlılıkları oluşturur, runtime'ı başlatır ve shutdown yönetir.

Import yönü: cmd -> transport/infrastructure/application -> domain. Domain hiçbir üst katmanı import etmez.

5. Configuration Sözleşmesi

Configuration yalnızca environment variable üzerinden yüklenir. Varsayılanlar geliştirme ergonomisi için kullanılabilir; secret veya deployment'a özgü adresler için güvenli olmayan varsayılan üretilmez.

Konu	Karar	Gerekçe
APP_ENV	local, test, staging, production	Desteklenmeyen değer startup hatasıdır.
HTTP_PORT	8080	1-65535 aralığında olmalıdır.
DATABASE_URL	Varsayılan yok	Boş olamaz; loglanamaz.
LOG_LEVEL	debug (local), deployment override	Logger kurulurken desteklenen seviyelerle doğrulanır.
WALK_WORKER_CONCURRENCY	2	Negatif olamaz; 0 ilgili worker işleme döngüsünü devre dışı bırakabilir.
OUTBOX_WORKER_CONCURRENCY	1	Negatif olamaz; 0 ilgili worker işleme döngüsünü devre dışı bırakabilir.
HTTP_READ_TIMEOUT	5s	Pozitif duration olmalıdır.
HTTP_WRITE_TIMEOUT	10s	Pozitif duration olmalıdır.
HTTP_SHUTDOWN_TIMEOUT	10s	Pozitif duration olmalıdır.

6. PostgreSQL Bağlantı Mimarisi

PostgreSQL erişimi github.com/jackc/pgx/v5 ve pgxpool ile gerçekleştirilir. database paketi bağlantı havuzunu üretir; global pool saklamaz ve süreç yaşam döngüsünü sahiplenmez.

```
package database

func Open(ctx context.Context, databaseURL string) (*pgxpool.Pool, error)
```

- Open, boş DATABASE_URL değerini reddeder.
- pgxpool.ParseConfig kullanılır; manuel URL parçalama yapılmaz.
- Milestone 0 için özel MaxConns, MinConns, MaxConnLifetime veya MaxConnIdleTime değeri tanımlanmaz. pgxpool varsayılanları kullanılır.
- Pool oluşturulduktan sonra başlangıç doğrulaması için Ping çağrılır. Ping başarısızsa pool kapatılır ve açıklayıcı hata döndürülür.
- Fonksiyon hata mesajlarına bağlam ekler ancak DATABASE_URL veya parola gibi secret değerleri içermez.
- Pool sahibi composition root'tur ve shutdown sırasında pool.Close() çağırır. Ayı bir Close wrapper zorunlu değildir.

- Paket init(), panic veya log.Fatal kullanmaz; tüm hatalar çağırana döner.
- Open fonksiyonu çağırının context deadline/cancellation kararına uyar; paket kendi gizli timeout'unu icat etmez.

M0-006 bağlayıcı karar: pgxpool varsayılanları + startup Ping + caller-owned Close. Bu görev repository, migration veya transaction helper içermez.

7. HTTP API Sınırları

- HTTP server yalnızca api binary'sinde başlatılır.
- Server read, write ve shutdown timeout değerleri configuration'dan alınır.
- Handler'lar transport DTO'larını domain entity olarak kullanmaz.
- Handler doğrudan repository veya SQL çağırır; application use-case çağırır.
- Health endpoint'leri ayrı bir görevde tanımlanır; readiness kritik bağımlılıkların kullanılabilirliğini yansıtır.
- HTTP error response secret, stack trace veya internal SQL ayrıntısı içermez.

8. Worker Mimarisi

- Her worker kendi cmd entrypoint'i ve composition root'una sahiptir.
- Concurrency config ile belirlenir; negatif değer startup hatasıdır, 0 worker döngüsünü devre dışı bırakabilir.
- İşlem context cancellation'a duyarlı olmalıdır.
- Bir işin başarısızlığı tüm süreci panic ile sonlandırmamalıdır; hata sınıflandırılır ve retry politikasına bırakılır.
- İşler at-least-once çalışabileceği için idempotency gereksinimi application/domain seviyesinde ele alınır.
- Worker polling interval, retry/backoff ve batch size ancak ilgili görev veya belgede açıkça tanımlandığında eklenir.

9. Transactional Outbox Modeli

Domain değişikliği ile yayımlanacak event aynı PostgreSQL transaction'ında kalıcılaştırılır. outbox-worker daha sonra yayımlanmamış kayıtları güvenli şekilde alır ve yayın durumunu günceller.

- Outbox kayıt şeması migration görevinde tanımlanır.
- Publish mekanizması ve broker seçimi ilgili milestone'da tanımlanana kadar varsayılmaz.
- Worker duplicate publish olasılığına dayanıklı tasarlanır; tüketiciler idempotent olmalıdır.
- Outbox tablosuna doğrudan transport katmanından yazılmaz.

10. Logging ve Observability

- Structured logging kullanılır; standart çıktı production uyumlu ana sink'tir.

- Her kayıt en az level, message, timestamp ve service alanlarını içermelidir.
- Request ID, trace ID, job ID ve aggregate ID mevcut olduğunda bağlamsal alan olarak eklenir.
- DATABASE_URL, authorization header, token, parola ve tam kişisel konum verisi loglanmaz.
- Library paketleri process exit kararı vermez; log.Fatal yalnızca composition root'ta dahi tercih edilmez, main hata koduyla çıkabilir.
- Metrics ve tracing sonraki görevlerde eklenebilir; logging görevi sırasında business instrumentation uydurulmaz.

11. Hata Modeli

- Domain errors kullanıcı veya iş kuralı anlamı taşır.
- Infrastructure errors teknik bağlamla wrap edilir ve errors.Is/errors.As zinciri korunur.
- Transport katmanı internal hatayı güvenli HTTP durumuna ve response modeline dönüştürür.
- Hata mesajları secret veya kişisel veri içermez.
- panic yalnızca geri dönülemez programlama invariant ihlalleri için düşünülebilir; normal kontrol akışı değildir.

12. Yaşam Döngüsü ve Graceful Shutdown

19. Config yüklenir ve doğrulanır.
20. Logger oluşturulur.
21. Database pool açılır ve Ping ile doğrulanır.
22. Repository/use-case/transport veya worker bağımlılıkları oluşturulur.
23. Runtime başlatılır.
24. SIGINT/SIGTERM beklenir.
25. Yeni iş kabulü durdurulur; HTTP için Shutdown, worker için cancellation uygulanır.
26. HTTP_SHUTDOWN_TIMEOUT sınırı içinde tamamlanması beklenir.
27. Database pool en son kapatılır.

Kapanış sahipliği: Kaynağı oluşturan composition root onu kapatır. Infrastructure paketi gizli goroutine veya global lifecycle yönetimi kurmaz.

13. Güvenlik

- Secret değerler environment veya deployment secret store üzerinden sağlanır.
- SQL injection'a karşı parametrik sorgular kullanılır; string birleştirme ile SQL üretilmez.
- Least privilege database kullanıcıları deployment katmanında uygulanır.
- Kullanıcı girdisi transport sınırında şekil ve boyut açısından doğrulanır; domain invariants ayrıca domain katmanında korunur.
- Konum verisi hassas kabul edilir; gereksiz loglama ve response yansıtması yapılmaz.

14. Test Stratejisi

- Domain ve application testleri hızlı, deterministik ve mümkün olduğunca infrastructure bağımsızdır.
- Configuration testleri t.Setenv ile izole edilir.
- Database Open fonksiyonu için salt unit test zorunlu değildir; gerçek bağlantı davranışı integration test ile doğrulanır.
- Integration testleri ayrı test database kullanır ve migration seviyesini açıkça yönetir.
- Testler production secret veya geliştirici local database'ine bağlanmamalıdır.
- Her görev en az gofmt, go test ./... ve go build ./... kontrollerini çalıştırır.

15. Migration ve Veri Erişimi

- Migration dosyaları backend/migrations altında, sıralı ve immutable tutulur.
- Uygulama startup'ında otomatik migration çalıştırılmaz; migration ayrı operasyonel adım veya araçtır.
- Repository interface'leri application katmanında, concrete pgx implementasyonları infrastructure katmanında bulunur.
- Transaction boundary use-case gereksinimine göre açıkça yönetilir; generic transaction helper erken eklenmez.
- PostGIS türleri ve sorguları ilgili domain/veri görevi gelmeden tahmin edilmez.

16. Bağımlılık Kuralları

Konu	Karar	Gerekçe
Go standard library	Her yerde izinli	Öncelikli temel.
pgx/v5 + pgxpool	Infrastructure database katmanında izinli	PostgreSQL standardı.
HTTP router/framework	Ancak ilgili görev açıkça seçtiğinde	Milestone 0'da varsayılmaz.
DI framework	Yasak	Manual composition tercih edilir.
ORM	Varsayılan olarak yasak	SQL ve pgx odaklı yaklaşım.
Global mutable state	Yasak	Test edilebilirlik ve lifecycle güvenliği.
Redis/RabbitMQ	Belge veya görev açıkça istemeden yasak	Erken altyapı genişlemesini engeller.

17. Milestone 0 Bağlayıcı Kararları

Konu	Karar	Gerekçe
------	-------	---------

M0-004 Configuration	Stdlib tabanlı env loader; DATABASE_URL zorunlu	Üçüncü taraf config dependency yok.
M0-005 Docker	postgres, api, walk-worker, outbox-worker; PostGIS 16-3.4	Redis ve reverse proxy yok.
M0-006 PostgreSQL Connection	pgxpool Open + Ping; defaults; caller Close	Repository/migration/ transaction helper yok.
Structured Logging	Ayrı görev	Database görevine logging implementation eklenmez.
CI Bootstrap	Milestone listesinde tanımlanan ayrı görev	Task numarası PROJECT_STATUS ve milestone belgesiyle birebir eşleşmelidir.

18. Kabul Kontrol Listesi

- ☐ Paket konumu ve import yönü bu belgeyle uyumlu.
- ☐ Yeni dependency açık mimari gerekçeye sahip.
- ☐ Global connection veya init-time side effect yok.
- ☐ Configuration zorunlu değerleri startup'ta reddediyor.
- ☐ DATABASE_URL ve diğer secret değerler loglanmıyor.
- ☐ Database pool pgxpool ile açılıyor ve Ping başarısızlığında kapatılıyor.
- ☐ Pool lifecycle composition root tarafından yönetiliyor.
- ☐ Graceful shutdown context cancellation'a uyuyor.
- ☐ Görev scope'u repository, migration veya sonraki milestone işlerini içermiyor.
- ☐ gofmt, go test ./... ve go build ./... başarılı.

Ek A — M0-006 Referans Uygulama Sözleşmesi

```
// Package database owns PostgreSQL pool construction only.
package database

func Open(ctx context.Context, databaseURL string) (*pgxpool.Pool, error) {
    // 1. reject blank URL
    // 2. pgxpool.ParseConfig(databaseURL)
    // 3. pgxpool.NewWithConfig(ctx, cfg)
    // 4. pool.Ping(ctx)
    // 5. on failure: pool.Close(); return wrapped error
    // 6. return pool; caller owns Close
}
```

Normatif ifade: Bu ek örnek isim ve davranış sözleşmesini tanımlar; tam kod değildir. Uygulama task kapsamındaki test edilebilir en küçük biçimde yapılmalıdır.

Sürüm Geçmişi

Sürüm	Tarih	Değişiklik	Durum
1.1	2026-07-20	Backend temel mimarisi ve M0-006 PostgreSQL bağlantı sözleşmesi tanımlandı.	Approved